

App Dev Project Report

1. Student Details

Name: K Mithra

Roll Number: 23F3001714

Email: 23f3001714@ds.study.iitm.ac.in

About Me: I am a student enrolled for the BS Degree at IIT Madras, currently doing Diploma in Programming and Diploma in Data Science side by side. I am really passionate about software, particularly the cybersecurity domain, embedded systems and like to work more with hard coding languages rather than UI designing.

2. Project Details

Project Title: Placement Portal Application

Problem Statement:

To build a Placement Portal Application (PPA) web application that allows Admin (Institute), Companies, and Students to interact with the system based on their roles.

Approach:

The project follows a client-server architecture where the frontend and backend communicate through REST APIs. The backend is built using Flask and Flask-RESTful, with SQLite managed through SQLAlchemy ORM and JWT-based authentication using Flask-JWT-Extended. Celery and Redis handle background tasks such as email notifications, report generation, and CSV exports, while Redis also provides API response caching for improved performance. The frontend is a Vue.js 3 Single Page Application served with Vite, styled using Bootstrap 5, and provides dedicated dashboards for Admin, Student, and Company users.

3. AI/LLM Declaration

I used **ClaudeAI** and **Gemini** to assist in understanding errors, understanding model schemas, mapping APIs, refining the UI and understanding the syntax of the languages used, specifically python. I also took assistance in generating small code snippets for the right syntax. **MetaAI** has been used to generate a few images.

The extent of AI/LLM usage is around **15-25%**, limited to **code suggestions and documentation formatting**. All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework for routing and logic
SQLAlchemy	Object Relational Mapper to manage attribute and queries
Celery & Redis	Background task queue for asynchronous processing, interview notifications, n reports, and CSV exports
Flask-JWT-Extended	Provides secure JWT-based authentication and authorization for students, comp and administrators
Flask-Mail	Email service for sending interview notifications, account-related emails, and n placement reports
Axios	API Management
Vue.js 3 & Vite	Frontend framework and build tool for developing a fast, reactive Single Application (SPA)
Bootstrap 5	Frontend styling and responsive design

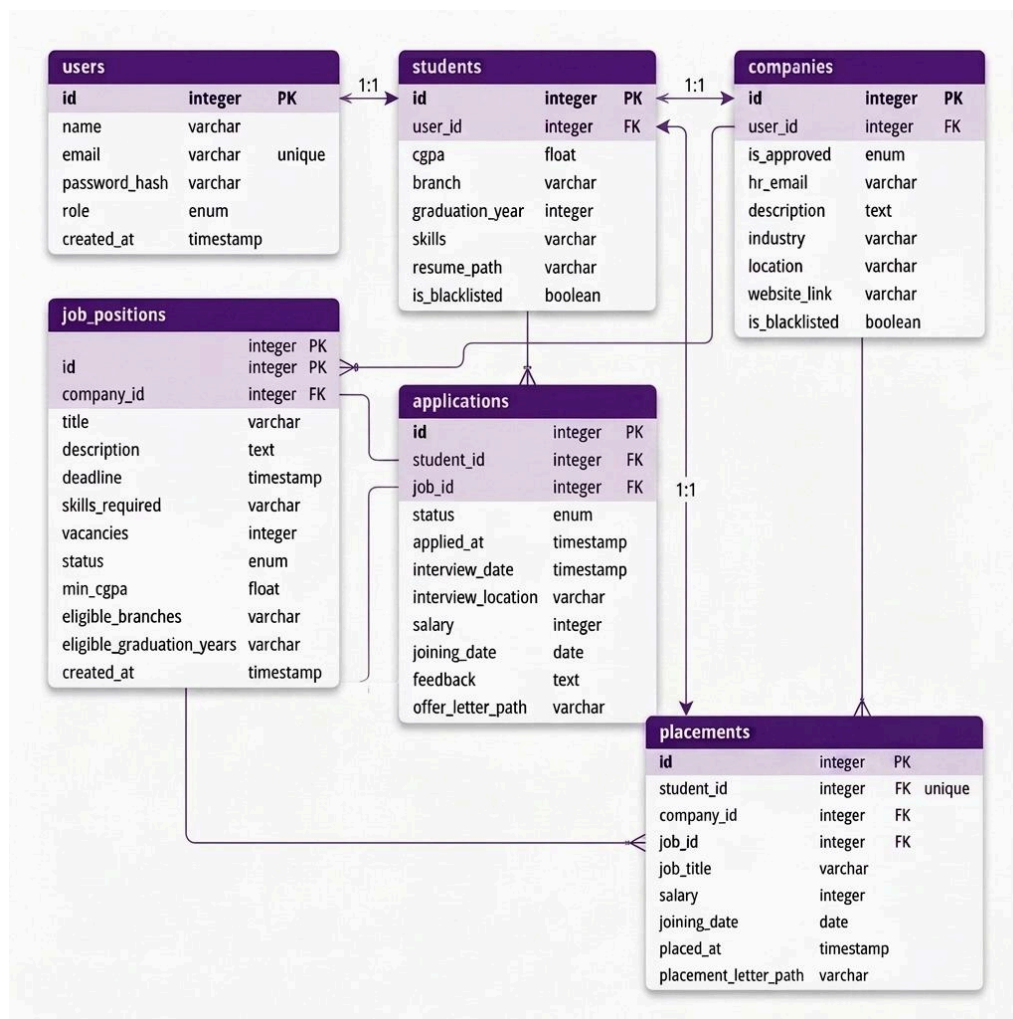
5. Database Schema / ER Diagram

Tables:

1. **User:** stores login credentials and role/type information (id, name, email, password_hash, role, created_at)
2. **Student:** stores student profile details and status (id , user_id , cgpa , branch , graduation_year , skills , resume_path , is_blacklisted)
3. **Company:** stores company profile and approval status (id , user_id , is_approved , hr_email , description , industry , location , website_link , is_blacklisted)
4. **JobPosition:** stores job postings created by companies (id , company_id , title , description , deadline , skills_required , vacancies , status , min_cgpa , eligible_branches , eligible_graduation_years , created_at)
5. **Application:** stores applications submitted by students for job openings (id , student_id , job_id , applied_at , status , interview_date , interview_location , salary , joining_date , feedback , offer_letter_path)
6. **Placement:** stores placement/hiring records once a student is placed (id , student_id , company_id , job_id , job_title , salary , joining_date , placed_at , placement_letter_path)

Relationships:

- **One-to-One (Unique link / single mapping)**
 - User → Student (via user_id)
 - User → Company (via user_id)
 - Student → Placement (via student_id - unique constraint)
- **One-to-Many**
 - Company → JobPosition
 - Company → Placement
 - Student → Application
 - JobPosition → Application
 - JobPosition → Placement
- **Many-to-One**
 - Student → User
 - Company → User
 - JobPosition → Company
 - Application → Student
 - Application → JobPosition
 - Placement → Student
 - Placement → Company
 - Placement → JobPosition



6. API Resource Endpoints

Endpoint URL	Method	Description
/api/auth/register/student	POST	Register student credentials and profile fields
/api/auth/register/company	POST	Register company profile, awaiting Admin validation
/api/auth/login	POST	Authenticate credentials and generate secure JWT
/api/student/profile	GET/PATCH	Fetch/update student details (CGPA, skills) or upload resume
/api/student/jobs	GET	Fetch active approved job positions
/api/student/job/<int:id>	GET	Fetch detailed eligibility criteria and info for a specific job
/api/student/job/apply	POST	Apply for a job position; verifies CGPA, branch eligibility
/api/student/applications	GET	List student applications and current workflow status
/api/student/application/<id>	GET/PATCH	View application details or accept/reject job offer
/api/student/export/csv	POST	Trigger asynchronous Celery task to export applications to CSV
/api/company/profile	GET/PATCH	Fetch/update company profile fields
/api/company/stats	GET	Get candidate ratios, open positions count, and placement rates
/api/company/job	POST	Create new job drive; default status set to PENDING admin review
/api/company/job/<id>	GET/PATCH	View job drive or open/close job listings manually
/api/company/applications	GET	Fetch all student applications across all company job drives
/api/company/application/<id>	PATCH	Update application status (shortlist, reject, release offer)
/api/company/application/<id>/interview	PATCH	Schedule interview date and location for student
/api/company/placements	GET	View all placements associated with company job drives
/api/company/export/csv	POST	Trigger background Celery task to email company placement data
/api/admin/stats	GET	Retrieve comprehensive portal statistics across students and companies
/api/admin/companies	GET	Fetch list of all registered companies (pending and approved)
/api/admin/company/<id>	PATCH	Approve/reject company registration status
/api/admin/company/<id>/blacklist-status	PATCH	Toggle blacklist status of a company (blocks active logins)
/api/admin/jobs	GET	List all job drive postings submitted by companies
/api/admin/job/<id>	PATCH	Approve/reject job postings (allows student visibility once approved)
/api/admin/students	GET	Fetch list of all registered student profiles
/api/admin/student/<id>	PATCH	Toggle student blacklist status (disallows application submissions)
/api/admin/placements	GET	Fetch list of all selected placements across the portal
/api/admin/trigger/interview-reminder	POST	Manual trigger for interview reminder Celery task
/api/admin/trigger/monthly-report	POST	Manual trigger for monthly report PDF generation Celery task
/api/admin/task/<task_id>	GET	Check background celery task execution state and result

YAML API Definition File:

Included separately in the submission ZIP as `api.yaml`.

7. Architecture and Features (optional)

```

placement-portal-application-v2-main/
|-- backend/
|   |-- routes/           # Flask-RESTful endpoints (admin, auth, company, student)
|   |-- services/         # Business logic (auth, application, pdf, stats, placement)
|   |-- static/           # Storage for user resumes, exported reports
|   |-- app.py            # Main Flask initialization, Celery beat & middleware config
|   |-- models.py         # SQLAlchemy model schemas (User, Student, Company, Jobs, Placements)
|   |-- tasks.py          # Celery background tasks (monthly reports, deadline alerts)
|   |-- requirements.txt   # Backend python package requirements
|-- frontend/
|   |-- src/
|       |-- views/        # Vue dashboard pages (Admin, Student, Company, login components)
|       |-- router/       # Vue Router mapping endpoints to views
|       |-- services/     # Axios API handlers for student, company, and admin endpoints
|       |-- App.vue        # Root Vue layout component
|       |-- main.js        # Vue instance entrypoint
|   |-- package.json      # Node dependencies (Vue, Axios, Bootstrap, Pinia)
|   |-- vite.config.js    # Vite configuration file

```

Implemented Features

- Authentication & Authorization – Role-based access control (RBAC) with secure session handling for Admin, Student, and Company users.
- Admin Dashboard – Interactive portal to manage blacklists, review pending registration requests, and inspect statistics.
- Student Profile & Job Search – Dynamic profile page for skill updates and resume uploads, alongside direct application filters based on eligibility.
- Company Console – Interface to list jobs, define criteria (min CGPA, branches), review applications, and toggle selection statuses.
- Automated Placement Cycle – Real-time application status flow (Applied, Shortlisted, Interview, Placed) and immediate placement records generation.
- Background Reporting – Periodic report preparation (e.g. monthly company metrics) executed asynchronously using Celery.

Additional Features

- CSV Data Exporter – Exporters to download tabular placement data as CSV formats.
- PDF Report Generation – Automatic generation and download of placement and selection reports in PDF format.

8. Video Presentation

Drive Link:

 **Mithra MAD-2 Project** *Project Video Link*